

RAPPORT D'AUDIT TECHNIQUE

HealthFlow Guinea

Analyse approfondie de l'architecture, de la securite et des performances du systeme d'information hospitalier.
Recommandations priorisees et feuille de route pour la mise en production.

ORGANISATION

DataSphere Innovation

DIRECTEUR

Sekouna KABA

DATE

Mai 2026

ANALYSE & RECOMMANDATIONS V1.0

Table des Matieres

1. Resume Executif	3
2. Analyse de l'Architecture	3
2.1 Stack Technique et Infrastructure	3
2.2 Forces Architecturales	4
2.3 Faiblesses Architecturales	4
3. Audit de Securite	5
3.1 Vulnerabilites Critiques (Niveau 1)	5
3.1.1 SEC-01 : OTP dans la reponse API	5
3.1.2 SEC-03 : Faux chiffrement base64	6
3.2 Vulnerabilites Elevees (Niveau 2)	6
3.3 Vulnerabilites Moyennes (Niveau 3)	6
4. Performance et Qualite du Code	7
4.1 Problemes de Performance	7
4.2 Qualite du Code et Dette Technique	7
5. FHIR R4 et Interoperabilite	8
5.1 Implementation FHIR R4	8
5.2 Integrations Nationales	8
6. Recommandations Priorisees	9
6.1 Actions immediates - Securite (0-2 semaines)	9
6.2 Actions a Court Terme - Architecture (2-8 semaines)	9
6.3 Actions a Moyen Terme - Production (2-6 mois)	10
7. Evolutions et Feuille de Route	11
7.1 Phase 8 : Securisation et Conformite	11

7.2 Phase 9 : Architecture Production-Ready	11
7.3 Phase 10 : Integrations Nationales Reelles	11
7.4 Phase 11 : Intelligence Artificielle et Telemedecine Avancee	11
7.5 Phase 12 : Deploiement National et Passage a l'Echelle	12
8. Recommandations DevOps et Infrastructure	12
8.1 Ameliorations Docker	12
8.2 Pipeline CI/CD	13
9. Conclusion et Perspectives	13

1. Resume Executif

HealthFlow Guinea est un systeme d'information hospitalier (SIH) ambitieux concu pour l'infrastructure sanitaire nationale de la Republique de Guinee. Developpe par DataSphere Innovation, sous la direction de Sekouna KABA, ce projet vise a fournir une plateforme complete couvrant la gestion des patients, les consultations cliniques, la telemedecine, les paiements Mobile Money, l'interopabilite HL7 FHIR R4, et les integrations nationales (DHIS2, SNIS, mTrac, SANTEP). Le systeme adopte une approche offline-first adaptee aux realites connectives de la Guinee, avec support PWA et IndexedDB pour le fonctionnement sans connexion internet.

Cependant, cet audit revele des vulnerabilites de securite critiques qui compromettent gravement la confidentialite des donnees de sante, l'integrite du systeme d'authentification, et la conformite reglementaire. Les principales preoccupations incluent un chiffrement côté client qui n'est qu'un encodage base64, un hachage de mots de passe non cryptographique, des OTP renvoyes dans les reponses API, et une validation d'identite basee sur des en-tetes HTTP falsifiables. Ce rapport presente une analyse detaillee de 12 vulnerabilites critiques, des recommandations priorisees pour la mise en production, et une feuille de route pour les evolutions futures du systeme.

Indicateur	Valeur	Appreciation
Modules fonctionnels	30+	Tres bon
Vulnerabilites critiques	6	CRITIQUE
Vulnerabilites elevees	5	ELEVEE
Vulnerabilites moyennes	3	MOYENNE
Routes API	30+	Bon volume
Couverture de tests	0%	CRITIQUE
Langues supportees	5	Excellent
Modeles Prisma	40+	Tres bon
Niveau de maturite production	Prototype avance	MOYENNE

Tableau 1 : Synthese des indicateurs cles du projet HealthFlow Guinea

2. Analyse de l'Architecture

2.1 Stack Technique et Infrastructure

Le projet repose sur Next.js 16.1.3 avec le App Router, React 19, TypeScript 5, Tailwind CSS 4, et shadcn/ui pour l'interface utilisateur. La persistance des données est assurée par Prisma ORM avec SQLite en développement et PostgreSQL provisionne via Docker Compose en production. L'état applicatif est géré par Zustand avec persistance localStorage, complète par IndexedDB pour le mode hors-ligne. La sécurité repose sur Next-Auth avec authentification Phone+OTP, un système RBAC à 8 rôles, et un RLS (Row-Level Security) au niveau applicatif. L'infrastructure Docker inclut PostgreSQL 16, Redis 7, et Nginx comme reverse proxy avec rate limiting.

2.2 Forces Architecturales

- **Modélisation domaine healthcare complet** : Les types FHIR R4, la matrice RBAC, le MPI (Master Patient Index), et les intégrations nationales (DHIS2, SNIS, mTrac) témoignent d'une connaissance approfondie du domaine santé guinéen. La couverture fonctionnelle est impressionnante avec plus de 30 modules couvrant l'ensemble du cycle de soins.
- **Conception offline-first** : L'architecture IndexedDB + Service Worker + file de synchronisation est correctement implémentée avec trois stratégies de cache (Cache-First pour les statiques, Network-First pour les API, Stale-While-Revalidate pour les pages). Cette approche est essentielle pour le contexte guinéen où la connectivité est intermittente.
- **Structure modulaire claire** : La séparation entre lib (logique métier), composants (UI), hooks (reactivité), et API routes (endpoints) suit les bonnes pratiques Next.js. Chaque module métier (télémedecine, pharmacie, laboratoire, etc.) est correctement isolé.
- **Localisation Guinée-spécifique** : Le support de 5 langues (Français, Anglais, Malinke, Soussou, Poular), les codes de zones sanitaires, les établissements de santé, et les systèmes nationaux de santé publique montrent un engagement réel dans l'adaptation locale.
- **Infrastructure Docker production-ready** : Le Dockerfile multi-stage, le docker-compose avec PostgreSQL et Redis, et le script de déploiement démontrent une vision de mise en production.

2.3 Faiblesses Architecturales

- **Router côté client uniquement** : L'application utilise un unique page.tsx avec basculement de vues via Zustand au lieu du routage fichier de Next.js. Cela rend le SEO impossible, les liens profonds non fonctionnels, et la navigation par URL inexistante. C'est un anti-pattern majeur pour une application de cette envergure.
- **Données de demo dans les constructeurs** : Chaque service initialise avec des données en dur dans ses constructeurs (~700 lignes dans data-store.ts). Cela rend la transition vers la production extrêmement difficile et pollue le code métier avec des données de test.
- **Tout en mémoire** : Les journaux d'audit, les sessions OTP, les échanges HIE, et les enregistrements MPI sont tous stockés en mémoire. Un redémarrage serveur entraîne une perte complète de ces données, ce qui est inacceptable pour un système de santé.
- **Incohérence Prisma SQLite vs PostgreSQL** : Le schéma Prisma définit SQLite mais le docker-compose provisionne PostgreSQL. Cette incohérence empêche le déploiement Docker de fonctionner correctement avec la base de données.

- **Double systeme i18n concurrent** : Un fournisseur React Context personnalise et next-intl coexistent sans integration. next-intl est configure en dur sur le francais cote serveur, ignorant la locale cote client.

3. Audit de Securite

L'audit de securite revele des vulnerabilites graves qui rendent le systeme actuel impropre a un deploiement en production dans un contexte de sante. Les donnees de sante sont classees donnees sensibles par la reglementation (equivalent RGPD/HIPAA), et toute fuite ou alteration pourrait avoir des consequences legales et ethiques severes. Cette section detaille les 12 vulnerabilites identifiees, classees par niveau de severite.

3.1 Vulnerabilites Critiques (Niveau 1)

ID	Vulnerabilite	Fichier	Impact
SEC-01	OTP renvoye dans la reponse API - tout appelant obtient le code OTP	api/auth/otp/route.ts	Contournement total de l'authentification
SEC-02	CORS autorise toutes les origines (Access-Control-Allow-Origin: *)	lib/api-utils.ts	Acces API depuis n'importe quel domaine
SEC-03	Chiffrement client = encodage base64 (btoa), reversible trivialement	lib/security.ts	Faussees donnees chiffrees lisibles par tous
SEC-04	Hachage mots de passe = DJB2 (non cryptographique, sans sel)	lib/security.ts	Mots de passe recuperables en clair
SEC-05	Role utilisateur depuis en-tetes HTTP falsifiables (x-user-role)	lib/api-middleware.ts	Escalade de privileges arbitraire
SEC-06	Protection CSRF desactivee dans secureApiHandler	lib/api-middleware.ts	Attaques CSRF sur toutes les routes

Tableau 2 : Vulnerabilites critiques identifiees

3.1.1 SEC-01 : OTP dans la reponse API

Le systeme d'authentification Phone+OTP presente une faille fondamentale : le code OTP genere est retourne directement dans le corps de la reponse HTTP de l'endpoint /api/auth/otp. Cela signifie que tout acteur capable d'appeler cette API (application malveillante, script, navigateur compromis) obtient immediatement le code OTP sans necessiter d'acces au telephone du destinataire. Cette vulnerabilite annule completement le benefice de l'authentification a deux facteurs. Dans un systeme de sante, un attaquant pourrait usurper l'identite de n'importe quel professionnel de sante et acceder aux dossiers medicaux des patients, prescrire des medicaments, ou modifier des donnees cliniques.

De plus, le stockage OTP en memoire (Map JavaScript) signifie que tous les OTP sont perdus lors d'un redemarrage serveur et ne sont pas partagees entre les instances en cas de montee en charge. La correction immediate consiste a ne jamais retourner l'OTP dans la reponse API, a l'envoyer exclusivement par SMS via un fournisseur tiers (Orange Money API, Twilio), et a utiliser Redis pour le stockage OTP avec expiration automatique.

3.1.2 SEC-03 : Faux chiffrement base64

La fonction encryptField() dans lib/security.ts utilise btoa(encodeURIComponent(data)) comme mecanisme de chiffrement cote client. L'encodage base64 n'est PAS du chiffrement : il s'agit d'un encodage reversible trivialement decode avec atob(). N'importe quel developpeur ou outil automatique peut decoder ces donnees en une seule ligne de code. Cette vulnerabilite est particulierement grave car elle cree une illusion de securite sans fournir aucune protection reelle. Les donnees de sante ainsi ' chifrees' sont en realite accessibles en clair a quiconque inspecte le trafic reseau ou le stockage local.

Ironiquement, le serveur dispose d'une implementation correcte avec encryptFieldServer() utilisant AES-256-CBC via le module crypto de Node.js. La correction consiste a deplacer toutes les operations de chiffrement vers le serveur et a utiliser Web Crypto API (crypto.subtle) pour le chiffrement cote client lorsque cela est absolument necessaire, avec des clesees separement.

3.2 Vulnerabilites Elevees (Niveau 2)

ID	Vulnerabilite	Fichier	Impact
SEC-07	Secrets d'auth codes en dur dans docker-compose (healthflow_secret_2024)	docker-compose.yml	Forgage de JWT possible
SEC-08	CSP autorise unsafe-inline et unsafe-eval	middleware.ts	Protection XSS significativement affaiblie
SEC-09	Token d'auth dans localStorage (accessible via XSS)	lib/auth-store.ts	Vol de session par XSS
SEC-10	Aucune verification JWT dans les routes API	Toutes les routes api/	Acces non authentifie aux donnees
SEC-11	Cle secrete NextAuth avec fallback hardcoded	lib/auth.ts	JWT forgable si variable absente

Tableau 3 : Vulnerabilites de niveau eleve

3.3 Vulnerabilites Moyennes (Niveau 3)

ID	Vulnerabilite	Fichier	Impact
SEC-12	ignoreBuildErrors: true masque les erreurs TypeScript	next.config.ts	Bugs silencieux en production
SEC-13	Toutes les regles ESLint desactivees	eslint.config.mjs	Aucune verification qualite automatisee
SEC-14	RLS applicative uniquement (non appliquee en BDD)	lib/rls.ts	Contournement possible via route non protegee

Tableau 4 : Vulnerabilites de niveau moyen

4. Performance et Qualite du Code

4.1 Problemes de Performance

L'audit de performance identifie plusieurs problemes qui afecteront significativement les temps de chargement et l'experience utilisateur en production, particulierement sur les connexions bas debit typiques en Guinee (2G/3G). Ces problemes doivent etre corriges avant toute mise en production car ils impactent directement la capacite du systeme a fonctionner dans son environnement cible.

Probleme	Detail	Impact
Chargement eager des composants	30+ composants de page importes dans page.tsx sans React.lazy() ni dynamic()	Bundle JS initial excessif, premier chargement lent
Donnees demo au chargement	~700 lignes de donnees patients creees a chaque import du module data-store	Memoire consomsee inutilement, latence au demarrage
Absence de pagination	Tous les patients, rendez-vous, etc. charges en memoire simultanement	Degradation progressive avec le volume de donnees
Prisma query logging permanent	log: ["query"] active dans toutes les configurations	Overhead I/O inutile en production

Tableau 5 : Problemes de performance identifies

4.2 Qualite du Code et Dette Technique

La qualite du code presente un paradoxe interessant : d'un cote, la modelisation metier et la couverture fonctionnelle sont excellentes, temoignant d'une bonne comprehension du domaine sante ; de l'autre, les pratiques de developpement (validation, tests, typage) sont insuffisantes pour un projet de cette envergure. Le fichier de configuration ESLint desactive virtuellement toutes les regles significatives

(no-explicit-any: off, no-unused-vars: off, exhaustive-deps: off), ce qui supprime la première ligne de défense contre les erreurs courantes. De même, TypeScript est configuré avec noImplicitAny: false et ignoreBuildErrors: true, créant une illusion de typage sans ses bénéfices réels.

L'absence totale de tests (0% de couverture) est le problème le plus préoccupant. Pour un système de santé où les erreurs peuvent avoir des conséquences vitales, cette absence est inacceptable. La dépendance Zod est présente dans package.json mais n'est utilisée dans aucune route API, laissant les endpoints sans validation d'entrée. Enfin, le nom du package reste nextjs_tailwind_shadcn_ts (nom du template par défaut), ce qui suggère que le projet n'a pas été initialisé avec une identité propre.

5. FHIR R4 et Interoperabilité

5.1 Implementation FHIR R4

L'implémentation FHIR R4 est l'un des points forts du projet, avec des définitions de types complètes couvrant Patient, Observation, DiagnosticReport, Encounter, MedicationRequest, Practitioner, Organization, Bundle et OperationOutcome. Le mapping bidirectionnel (patientToFHIR / fhirToPatient) est correctement implémenté, et les systèmes de codes Guinée-spécifiques (ID national, zones de santé, établissements, formulaire national de médicaments, carte SANTEP) sont un atout majeur pour l'adoption du système dans le contexte national.

Cependant, l'implémentation actuelle reste conceptuelle : le endpoint /api/fhir/[...path] existe mais toutes les données sont en mémoire ou de démonstration. Il n'y a pas de serveur FHIR réel avec persistance en base de données, pas d'implémentation des paramètres de recherche FHIR, pas d'opération \$validate contre les profils HL7 officiels, et les signatures des Bundles ne contiennent aucune donnée cryptographique. Pour atteindre la conformité FHIR, le système doit implémenter un serveur FHIR complet avec validation, recherche, et persistance.

5.2 Integrations Nationales

Système	Statut	Écarts
DHIS2	Structures de données définies	Pas de client API réel, pas de push de données
SNIS	Indicateurs définis avec cibles et tendances	Données demo uniquement, pas d'intégration API
mTrac	Alertes et rapports hebdomadaires définis	Pas de client API mTrac
SANTEP	Structure carte électronique avec QR/NFC	Pas de système d'émission/vérification
HIE	Échange inter-établissements le plus complet	En mémoire uniquement, pas d'appels réseau

Tableau 6 : Statut des integrations nationales

6. Recommandations Priorisees

6.1 Actions Immediates - Securite (0-2 semaines)

Ces actions sont indispensables avant toute mise en production ou demonstration publique du systeme. Elles concernent exclusivement la correction des vulnerabilites de securite critiques et elevees identifiees dans la section 3. Leur non-resolution expose le systeme a des risques juridiques et ethiques majeurs dans le contexte des donnees de sante.

Prio rite	Action	Detail technique	Ref
P0	Supprimer l'OTP de la reponse API	Envoyer uniquement par SMS via fournisseur tiers, stocker dans Redis avec TTL	SEC-01
P0	Restreindre CORS aux domaines autorises	Remplacer * par la liste des domaines de confiance (frontend, admin)	SEC-02
P0	Remplacer le faux chiffrement par crypto.subtle	Deplacer le chiffrement serveur (AES-256-CBC existant), Web Crypto API cote client	SEC-03
P0	Remplacer simpleHash par bcrypt/argon2	Utiliser bcryptjs (compatible Edge) ou argon2 pour le hachage des mots de passe	SEC-04
P0	Verifier JWT dans le middleware API	Utiliser getSession de NextAuth, ne jamais faire confiance aux en-tetes clients	SEC-05
P0	Reactiver la protection CSRF	Decommenter et activer la validation CSRF dans secureApiHandler	SEC-06
P1	Rendre NEXTAUTH_SECRET obligatoire	Supprimer le fallback hardcoded, faire echouer le demarrage si absent	SEC-11
P1	Renforcer la CSP	Supprimer unsafe-inline et unsafe-eval, utiliser des nonces ou hashes pour les scripts	SEC-08

Tableau 7 : Actions immediates de securite

6.2 Actions a Court Terme - Architecture (2-8 semaines)

Ces actions visent a transformer le prototype en une application production-ready. Elles concernent les corrections architecturales fondamentales qui permettront au systeme de passer a l'echelle et de fonctionner de maniere fiable dans un environnement de production.

Action	Objectif	Approche recommandee
Migrer vers le routage fichier Next.js	SEO, liens profonds, navigation par URL	Convertir les vues Zustand en routes /app avec layout.tsx par section
Ajouter la validation Zod aux routes API	Protection contre les injections et donnees invalides	Creer des schemas Zod pour chaque endpoint, valider dans le middleware
Extraire les donnees demo des constructeurs	Separation code metier / donnees de test	Scripts de seed Prisma, feature flag DEMO_MODE, variables d'environnement
Code splitting avec React.lazy()	Reduction du bundle initial, chargement a la demande	Dynamic imports pour chaque module page, Suspense boundaries
Aligner Prisma sur PostgreSQL	Coherence avec l'infrastructure Docker	Changer provider en postgresql, adapter le schema, tester les migrations
Implementer un serveur FHIR avec persistance	Conformite HL7 FHIR R4	Stocker les ressources FHIR en BDD, implementer les parametres de recherche

Tableau 8 : Actions a court terme pour l'architecture

6.3 Actions a Moyen Terme - Production (2-6 mois)

- **Vérification de session cote serveur** : Implementer getSession dans chaque route API pour verifier l'authenticite des requetes. Utiliser les tokens JWT avec rotation et expiration courte (15 minutes pour l'accès, 7 jours pour le refresh).
- **Row-Level Security au niveau base de donnees** : PostgreSQL offre des politiques RLS natives qui garantissent que chaque utilisateur ne peut acceder qu'aux donnees de son etablissement et de son role. Cela fournit une defense en profondeur au-dela du filtrage applicatif.
- **Redis pour le stockage OTP et le rate limiting** : Deja provisionne dans docker-compose, Redis doit etre utilise pour les OTP (avec TTL automatique), les sessions, et le rate limiting distribue entre les instances.
- **Clients API reels pour DHIS2/SNIS/mTrac** : Implementer des clients HTTP avec retry, circuit breaker, et gestion des erreurs pour chaque integration nationale. Utiliser les SDK officiels lorsqu'ils existent.
- **Suite de tests complete** : Objectif de couverture minimum 80% avec tests unitaires (Vitest), tests d'integration (Testing Library), et tests E2E (Playwright). Prioriser les tests sur les flux critiques : authentification, prescription, paiement Mobile Money.

- **Chiffrement de bout en bout pour le HIE** : Les échanges de données de santé entre établissements doivent être chiffrés en transit (TLS 1.3) et au repos (AES-256), avec gestion des clés par établissement et audit trail des accès.

7. Evolutions et Feuille de Route

7.1 Phase 8 : Sécurisation et Conformité

La Phase 8 constitue la priorité absolue et doit précéder toute autre évolution. Elle se concentre sur la correction de toutes les vulnérabilités identifiées dans cet audit et la mise en conformité du système avec les exigences réglementaires applicables aux données de santé. Cette phase inclut la refonte du système d'authentification, l'implémentation du chiffrement réel, la vérification JWT dans toutes les routes API, la RLS au niveau base de données, et l'audit trail persistant. La durée estimée est de 4 à 6 semaines avec un objectif de conformité aux normes équivalentes à HIPAA et RGPD pour les données de santé.

7.2 Phase 9 : Architecture Production-Ready

La Phase 9 transforme le prototype en une application production-ready. Les chantiers principaux incluent la migration vers le routage fichier Next.js (remplacement du routage Zustand par les routes /app), la mise en place de la validation Zod sur toutes les routes API, l'extraction des données de demo vers des scripts de seed, et l'implémentation du code splitting avec React.lazy() pour réduire le bundle initial. Cette phase inclut également la migration vers PostgreSQL avec les politiques RLS natives, l'implémentation d'un vrai serveur FHIR avec persistance, et la configuration d'un pipeline CI/CD avec tests automatiques. La durée estimée est de 6 à 10 semaines.

7.3 Phase 10 : Integrations Nationales Réelles

La Phase 10 matérialise les intégrations nationales actuellement définies conceptuellement. Le premier chantier est le connecteur DHIS2 avec synchronisation automatique des indicateurs sanitaires, suivi du connecteur SNIS pour la génération et le push des rapports mensuels, du connecteur mTrac pour la surveillance épidémiologique en temps réel, et du système SANTEP pour l'émission et la vérification des cartes de santé électroniques. Le Health Information Exchange (HIE) doit être implémenté avec un chiffrement de bout en bout et une gestion des consentements patients conforme aux réglementations guinéennes. La durée estimée est de 8 à 12 semaines, avec des livraisons incrémentales par connecteur.

7.4 Phase 11 : Intelligence Artificielle et Télémédecine Avancée

La Phase 11 enrichit les capacités d'IA du système. L'assistant diagnostique actuel doit être connecté à de vrais modèles d'IA (via API sécurisée) avec validation clinique, les interactions médicamenteuses doivent s'appuyer sur la base de données nationale des médicaments, et la surveillance épidémiologique doit utiliser des modèles prédictifs pour les alertes précoces. La télémédecine doit évoluer vers la

videoconsultation avec WebRTC, le partage d'ecran pour l'imagerie medicale, et l'integration avec lesappareils de diagnostic portables (echographes, tensiometres connectes). Cette phase inclut egalementl'implementation de l'IA pour le tri automatique des urgences et la detection des epidemies. La dureeestimee est de 10 a 16 semaines.

7.5 Phase 12 : Deploiement National et Passage a l'Echelle

La Phase 12 constitue le deploiement a l'echelle nationale. Les chantiers incluent l'architecture multi-tenant pour supporter les 8 zones sanitaires et les etablissements de sante de toutes les regions, l'optimisation pour les connexions bas debit (2G/3G) avec compression et progressive loading, le deploiement sur infrastructure cloud guineenne ou regionale avec haute disponibilite, et la formation des agents de sante communautaire (ASC) avec des interfaces simplifiees en langues locales. Le systeme doit pouvoir supporter simultanement des milliers d'utilisateurs sur l'ensemble du territoire national avec une disponibilite de 99.5%. Cette phase inclut egalement la mise en place d'un systeme de support technique national et la certification officielle du systeme par les autorites de sante guineennes. La duree estimee est de 12 a 20 semaines.

Phase	Thematique	Duree estimee	Prerequis
Phase 8	Securisation et Conformite	4-6 semaines	Aucun
Phase 9	Architecture Production-Ready	6-10 semaines	Phase 8 terminee
Phase 10	Integrations Nationales Reelles	8-12 semaines	Phase 9 terminee
Phase 11	IA et Telemedecine Avancee	10-16 semaines	Phase 9 terminee (parallele possible avec Phase 10)
Phase 12	Deploiement National	12-20 semaines	Phases 10-11 validees

Tableau 9 : Feuille de route des evolutions

8. Recommandations DevOps et Infrastructure

8.1 Ameliorations Docker

L'infrastructure Docker existante est un bon point de depart mais necessite des ajustements critiques avant le deploiement en production. Les identifiants par defaut cods en dur dans le docker-compose.yml (healthflow_secret_2024, hf-guinea-secret-change-in-production-2024) doivent etre remplaces par des variables d'environnement injectees au deploiement, sans valeur par defaut. Les ports PostgreSQL (5432) et Redis (6379) exposes sur l'hote constituent un risque de securite en production et doivent etre restreints au reseau Docker interne. Le pgAdmin en developpement ne doit jamais avoir PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED=False.

- **Secret management** : Utiliser Docker Secrets ou un coffre-fort de secrets (HashiCorp Vault) pour gerer les mots de passe et cles API. Ne jamais stocker de secrets dans le code source ou les fichiers docker-compose.
- **Health checks** : Ajouter des health checks pour PostgreSQL, Redis, et Nginx dans le docker-compose. Le health check existant sur /api/health est un bon debut.
- **Resource limits** : Definir des limites de memoire et CPU pour chaque conteneur (deploy.resources dans docker-compose) pour eviter qu'un service ne consomme toutes les ressources.
- **Logging centralise** : Configurer un pilote de logging (json-file avec rotation, ou Loki) pour eviter que les logs ne remplissent le disque.
- **Backup automatise** : Implementer des sauvegardes automatiques de PostgreSQL avec pg_dump programmees et stockees sur un stockage externe chiffré.

8.2 Pipeline CI/CD

Un pipeline CI/CD est indispensable pour garantir la qualite continue du code et la fiabilite des deploiements. Le pipeline recommande comprend les etapes suivantes : analyse statique (ESLint avec regles strictes, TypeScript strict mode), tests unitaires et d'integration avec couverture minimale de 80%, scan de securite des dependances (npm audit, Snyk), build et test de l'image Docker, deploiement automatise en staging avec tests de smoke, et deploiement en production apres validation manuelle. Les outils recommandes sont GitHub Actions ou GitLab CI pour l'orchestration, SonarQube pour l'analyse de qualite, et Trivy pour le scan des images Docker.

9. Conclusion et Perspectives

HealthFlow Guinea est un projet ambitieux et prometteur qui demontre une comprehension profonde des enjeux sanitaires guineens et une couverture fonctionnelle impressionnante. La modelisation FHIR R4, les integrations nationales (DHIS2, SNIS, mTrac, SANTEP), le support multilingue avec les langues locales, et l'approche offline-first sont des atouts majeurs qui positionnent ce systeme comme une solution potentiellement transformatrice pour le systeme de sante guineen.

Cependant, l'audit revele que le systeme est actuellement au stade de prototype avance et ne peut en aucun cas etre deploie en production sans la correction prealable des vulnerabilites de securite critiques. Les 6 vulnerabilites critiques identifiees (OTP dans la reponse API, CORS ouvert, faux chiffrement, hachage non cryptographique, en-tetes falsifiables, CSRF desactive) constituent des risques majeurs pour la confidentialite des donnees de sante et la securite du systeme. Leur correction doit etre la priorite absolue, avant toute autre evolution fonctionnelle.

La feuille de route proposee en 5 phases (Securisation, Architecture, Integrations, IA, Deploiement national) offre un chemin structure vers un systeme production-ready et deployable a l'echelle nationale. Avec un investissement estime de 40 a 64 semaines de developpement, HealthFlow Guinea a le potentiel de devenir le systeme d'information hospitalier de reference pour la Guinee et un modele pour les pays d'Afrique de l'Ouest confrontes aux memes defis sanitaires et technologiques.